rpc list and call methods to test out the service. The sample calls are shown below:

```
$ grpcli -f osfy_stats.proto --ip=127.0.0.1 --port=50000 -i
Package: osfy_stats
Service: OSFYStatsService
Host: 127.0.0.1
Port: 50000
Secure: No
[grpc+insecure://127.0.0.1:50000]# rpc list
toparticles(TopArticlesRequest) {
  return TopArticlesResponse;
}
articleStats(ArticleStatsRequest) {
  return ArticleStatsResponse;
}
[grpc+insecure://127.0.0.1:50000]# rpc call articleStats
{"id":1}
Info: Calling articleStats on OSFYStatsService
Response:
{
  "id": 1,
  "numViews": 1000,
  "numLikes": 30,
  "numComments": 5
}
[grpc+insecure://127.0.0.1:50000]#
```

Consuming the gRPC service

Now that we have tested our service, we can write our client code as shown below. The code is similar in the sense that we load the proto file first and then create a client to the server that is running. Once the client is connected, we can directly invoke the service methods as shown below:

```
const grpc = require('grpc');

const proto = grpc.load('osfy_stats.proto');

const client = new proto.osfy_stats.OSFYStatsService('localho
st:50000', grpc.credentials.createInsecure());

client.toparticles({"numResults":2}, (error, response) => {
  if (!error) {
    console.log("Total Articles: " + response.articles.
length);
    for (article of response.articles) {
      console.log(article.id + " " + article.title + " " +
article.url);
    }
  } else {
    console.log("Error:", error.message);
  }
});

client.articleStats({"id":2}, (error, response) => {
  if (!error) {
    console.log("Article ID : " + response.id + " Views :
" + response.numViews + " Likes : " + response.numLikes + "
Comments : " + response.numComments );
  } else {
    console.log("Error:", error.message);
  }
});
```

You can execute the client and see that the two actions available by the service are invoked. The output is shown below:

```
$ node client.js
Article ID : 2 Views : 1000 Likes : 30 Comments : 5
Total Articles: 2
20000 T1 URL1
20001 T2 URL2
```

Other language bindings

We have seen how we can define the service interface via the ProtocolBuffers format, and then used Node.js to develop both the server side and client side bindings. The advantage of gRPC is that your server side could be implemented in a specific language, say Node.js, but the client bindings could be in another language, like Python, Java or other supported bindings.

To see the list of languages supported, check out the documentation page (*http://www.grpc.io/docs/*) and select a language of your choice.

Modern architectures suggest breaking up monolithic applications into multiple micro-services and to compose your applications via those services. This results in an explosion of inter-service calls and it is important that latency, which is one of the key factors to consider in providing a high performance system, is addressed. gRPC provides both an efficient wire transfer protocol and multiple language bindings that make this a possibility. With the recent adoption of gRPC as a top level project in the Cloud Native Computing Foundation, we are likely to see an increase in developers exploring and using this across a wide range of projects. END

References

[1] gRPC.io home page: *www.grpc.io*
[2] gRPC Documentation: *http://www.grpc.io/docs/*
[3] Protocol Buffers Documentation: *https://developers.google.com/protocol-buffers/docs/overview*
[4] gRPC Community: *http://www.grpc.io/community/*
[5] Cloud Native Computing Foundation: *https://www.cncf.io/*

By: Romin Irani

The author has been working in the software industry for more than 20 years. His passion is to read, write and teach about technology, in order to help developers succeed. He blogs at *www.rominirani.com*.